Data Types

- Data type:
  - A data type in Python refers to the type or category of a value that a variable holds.
  - It tells the kind/classification of data stored in a variable.
- 9 Data Types in Python:
  1. int
  2. float
  3. str
  4. list
  5. tuple
  6. dict
  7. bool
  8. None
  9. set ❌ (not in the syllabus)

1. int (Integer):

- Whole numbers (no decimal point)
- Includes positive, negative & zero
- Immutable data type
  - ➔ E.g: i = 22
    value = 81
    a = -73
    x = -144

2. float (Floating number):

- Numbers with decimal point
- Includes positive and negative numbers
- Immutable data type
  - ➔ E.g: f = 98.3
    C = 32.11

Num = -59.7

fact = -120.24

3. str (String/Text):

- Collection (sequence) of characters (Alphabets, Special characters, Alphanumeric, numbers)
- Enclosed in quotes (4 types of quotes):
  - ➔ ' ' - single quotes
  - ➔ " " - double quotes
  - ➔ ''' ''' - triple single quotes
  - ➔ """ """ - triple double quotes

Multi-line strings

Single line strings

- Immutable data type
  - ➔ E.g: text = 'word123'

    name = "Python"

    year = '2026'

    sent = '''This is a sentence.

    This is a multi-line string.'''

    stmt = """This is a multi-line string.

    This uses triple double quotes."""

⚠ Note: Any value (stored in a variable) containing letters or special characters MUST always be enclosed within quotes.

4. list (List):

- Collection (sequence/ordered) of elements of different data types
- Enclosed in square brackets [ ]
- Mutable data type
- Each element is separated by a comma (,)
  - ➔ E.g: lst = [3, 5, 1, 9, 2]

    fruits = ['Apple', 'Banana', 'Jackfruit', 'Mango']

    mxdlst = [22, 77.3, 'text', [1,2,3], (66,22,11), True]

5. tuple (Tuple):

- Collection (sequence/ordered) of elements of different data types
- Enclosed in parentheses ( )
- Immutable data type
- Each element is separated by a comma (,)

➔ E.g: tpl = (3, 5, 1, 9, 2)

fruits = ('Apple', 'Banana', 'Jackfruit', 'Mango')

mxdtpl = (22, 77.3, 'text', [1,2,3], (66,22,11), True)

6. dict (Dictionary):

- Collection of elements of different data type in the form of 'key value' pair
- Enclosed in curly brackets { }
- Mutable data type
- Each element consists of key and value
- Key and value are separated by a colon (:)
- Key: Value and Key: Value are separated by a comma (,)

➔ E.g: dct = {1:100, 'name':'Python', 'mark':98.7}

d = {'rno':[1,2,3], 'name':['A','B','C'], 'mark':(77.1,53,92.4)}

⚠ Note: 👉 In a dict, 2 keys cannot have the same name (must be unique)

👉 int, float, str, tuple, bool ⟶ allowed as keys ✓

👉 int, float, str, tuple, list, dict, bool ⟶ allowed as values ✓

👉 list & dict data types cannot be set as the key in a dict

(as they are mutable types)

7. bool (Boolean):

- Represents logical values
- Only 2 values: True / False
- Immutable data type

8. None (Null):

- Represents no value/null value
- Data type: NoneType

📘 Data Types & Built-in Functions

1. Data Types

In Python, everything is an object, and every object has a data type. Data types are essentially classes, and variables are instances (objects) of those classes.

A. Number

Used to store numeric values. They are **immutable**.

1. Integer (`int`)

- **Description:** Represents whole numbers, positive or negative, without a decimal point.
- **Examples:** `10` , `-5` , `0` , `12345` .
- **Note:** Integers in Python 3 have unlimited length (only limited by system memory).

2. Floating Point (`float`)

- **Description:** Represents real numbers with a decimal point. They can also be represented in scientific notation (e.g., `1.2e3`).
- **Examples:** `3.14` , `-0.001` , `2.0` , `1.5e2` (which is 150.0).
- **Note:** Floating-point numbers are approximate values due to the way they are stored in memory.

3. Complex (`complex`)

- **Description:** Represents complex numbers with a real and an imaginary part. The imaginary part is denoted by `j` or `J` .
- **Examples:** `3 + 4j` , `5j` , `2 - 1j` .
- **Note:** Access real part via `z.real` and imaginary part via `z.imag` (where `z` is the complex variable).

B. Boolean (`bool`)

- **Description:** Represents one of two truth values: `True` or `False`. It is a subclass of `int`, where `True` acts like 1 and `False` acts like 0.
- **Usage:** Used for logical operations and conditional statements.
- **Examples:** `is_registered = True`, `is_valid = 3 > 2` (evaluates to `True`).

C. Sequence

A sequence is an ordered collection of items, where each item can be accessed via its index.

1. String (`str`)

- **Description:** Represents a sequence of characters (alphanumeric, symbols, etc.) enclosed within single, double, or triple quotes.
- **Examples:** `'Hello'`, `"CS Class"`, `'''Multi-line String'''`.
- **Nature:** IMMUTABLE (cannot be changed after creation).

2. List (`list`)

- **Description:** Represents an ordered, mutable collection of items. Items can be of different data types. Defined using square brackets `[]`.
- **Examples:** `[1, 2, 3]`, `['a', True, 10.5]`, `[]` (empty list).
- **Nature:** MUTABLE (can be changed after creation – add, remove, or modify elements).

3. Tuple (`tuple`)

- **Description:** Represents an ordered, immutable collection of items. Defined using parentheses `()`.
- **Examples:** `(1, 2, 3)`, `('a', True, 10.5)`, `()` (empty tuple).
- **Nature:** IMMUTABLE (cannot be changed after creation).

D. None (`NoneType`)

- **Description:** Represents the absence of a value or a null value. It is not the same as `0`, `False`, or an empty string.
- **Usage:** Often used to initialize a variable or to signify that a function has no return value.
- **Example:** `result = None`

E. Mapping (`dict` – Dictionary)

- **Description:** Represents an unordered, mutable collection of key-value pairs. Keys must be unique and immutable (e.g., strings, numbers, tuples), while values can be of any data type and are accessed via their key. Defined using curly braces `{}`.
- **Examples:** `{'name': 'Alice', 'age': 25}`, `{}` (empty dictionary).
- **Nature:** MUTABLE (can be changed after creation – add, remove, or update key-value pairs).

F. Mutable vs Immutable Data Types

This is a crucial concept in Python.

Feature	MUTABLE	IMMUTABLE
Definition	Objects whose state (value) can be changed after creation.	Objects whose state (value) **cannot** be changed after creation.
Memory	When modified, the object remains in the same memory location, and its content is updated.	When modified, a new object is created in a new memory location.
Data Types	`list`, `dict`, `set` (not in syllabus but good to know)	`int`, `float`, `complex`, `bool`, `str`, `tuple`

G. Dynamic Typing

Python is a **dynamically typed** language. This is one of its most important and beginner-friendly features.

What is Dynamic Typing?

- In dynamically typed languages, the **data type of a variable is determined at runtime** (during program execution), not at compile time.
- You **do not need to declare** the data type of a variable before using it. The interpreter infers the type based on the value assigned to it.

- The **same variable can hold values of different data types** at different points in the program.

Key Characteristics

- **No explicit type declaration:** Unlike languages like C, C++, or Java, you don't write `int x = 5`; you simply write `x = 5`.
- **Type is associated with the object, not the variable:** The variable is just a name (or reference) that points to an object in memory. The object itself knows its type.
- **Type can change:** A variable can be reassigned to a completely different data type.

```
# Variable 'x' starts as an integer
x = 10
print(x, type(x))  # Output: 10 <class 'int'>

# Same variable 'x' is now assigned a string
x = "Hello"
print(x, type(x))  # Output: Hello <class 'str'>

# Now 'x' becomes a floating-point number
x = 3.14
print(x, type(x))  # Output: 3.14 <class 'float'>

# Now 'x' becomes a boolean
x = True
print(x, type(x))  # Output: True <class 'bool'>

# Now 'x' becomes a list
x = [1, 2, 3]
print(x, type(x))  # Output: [1, 2, 3] <class 'list'>
```

---

2. Common Built-in Functions

Built-in functions are pre-defined functions in Python that are always available for use.

1. `eval()`

- **Purpose:** Evaluates a string expression and returns the result. It effectively treats the string as a Python expression and executes it.
- **Parameter(s):**
  - `source` : (Compulsory) The string expression to be evaluated.
- **Description:** Parses the string argument and compiles it to Python bytecode, which is then executed.

```
# Example 1: convert string input
value = eval(input("Enter a value"))
print(value)  # Output: 57.12 -- if the input is 57.12 float
value
print(type(value))  # Output: <class 'float'>

# Example 2: arithmetic expression
x = 10
result = eval("x * 2")  # Evaluates 10 * 2
print(result)  # Output: 20
```

2. `type()`

- **Purpose:** Returns the data type (class) of the argument passed to it.
- **Parameter(s):**
  - `object` : (Compulsory) The object whose type you want to know.
- **Description:** Returns a type object that tells you the class of the given object.

```
a = 10
b = 3.14
c = "Hello"
d = [1, 2]
e = (1, 2)
f = {'key': 'value'}

print(type(a))  # Output: <class 'int'>
print(type(b))  # Output: <class 'float'>
print(type(c))  # Output: <class 'str'>
print(type(d))  # Output: <class 'list'>
```

```
print(type(e))  # Output: <class 'tuple'>
print(type(f))  # Output: <class 'dict'>
```

3. `id()`

- **Purpose:** Returns the unique identifier (memory address) of an object.
- **Parameter(s):**
  - `object` : (Compulsory) The object whose memory address you need.
- **Description:** Every object in Python is stored at a specific memory location. The `id()` function returns this address as an integer. This is helpful to understand the concept of mutability and how Python handles variables.

```
x = 10
print(id(x))  # Output: Some memory address like 140735143830864

y = x  # y now points to the same memory location as x
print(id(y))  # Output: Same memory address as x

list1 = [1, 2]
print(id(list1))  # Output: e.g., 140281465641536

list2 = list1
print(id(list2))  # Output: Same as list1's address
```

4. `chr()`

- **Purpose:** Returns the character corresponding to a given Unicode code point (integer).
- **Parameter(s):**
  - `i` : (Compulsory) An integer (within the valid Unicode range, 0 to 1,114,111).
- **Description:** Converts an integer to its character representation. For example, 65 maps to 'A'.

```
print(chr(65))  # Output: A
print(chr(97))  # Output: a
```

```
print(chr(48))  # Output: 0 (zero)
print(chr(8364))  # Output: € (Euro symbol)
```

5. `ord()`

- **Purpose:** Returns the Unicode code point (integer) for a given character. It is the inverse of `chr()`.
- **Parameter(s):**
  - `c`: (Compulsory) A string of length one (a single character).
- **Description:** Converts a character to its integer representation.

```
print(ord('A'))  # Output: 65
print(ord('a'))  # Output: 97
print(ord('0'))  # Output: 48
print(ord('€'))  # Output: 8364
```

6. `len()`

- **Purpose:** Returns the number of items (length) of an object. The object must be a sequence type (string, list, tuple) or a collection (dictionary).
- **Parameter(s):**
  - `s`: (Compulsory) The sequence or collection object.
- **Description:** Counts the number of elements.
  - For strings: Counts the number of characters.
  - For lists/tuples: Counts the number of elements.
  - For dictionaries: Counts the number of key-value pairs.

```
text = "Computer Science"
print(len(text))  # Output: 16 (Spaces count too!)

my_dict = {'name': 'John', 'age': 30, 'city': 'NYC'}
print(len(my_dict))  # Output: 3 (Number of keys)
```

7. `sum()`

- **Purpose:** Adds all the items in an iterable (like a list or tuple) and returns the total sum.
- **Parameter(s):**
  - `iterable` : (Compulsory) A sequence of numbers (list, tuple, etc.) to be summed.
  - `start` : (Optional) A value to add to the sum. Default is 0.
- **Description:** Calculates the sum of all elements in the iterable. The elements must be numeric (int, float).

```
numbers = [10, 20, 30, 40]
total = sum(numbers)
print(total)  # Output: 100

# With start parameter
total_with_start = sum(numbers, 5)
print(total_with_start)  # Output: 105 (100 + 5)

# Sum of float values
float_nums = [1.5, 2.7, 3.2]
print(sum(float_nums))  # Output: 7.4
```

8. `min()`

- **Purpose:** Returns the smallest item from an iterable or from two or more arguments.
- **Parameter(s):**
  - `arg1, arg2, ...` OR `iterable` : (Compulsory) Can be multiple arguments or a single iterable.
- **Description:** Finds the minimum value. Works with numbers, strings (lexicographic comparison), and other comparable data types.

```
# With iterable
numbers = [45, 12, 78, 23, 9, 56]
print(min(numbers))  # Output: 9

# With multiple arguments
print(min(100, 200, 50, 75, 150))  # Output: 50
```

```
# With strings (alphabetical order)
words = ["apple", "banana", "cherry", "date"]
print(min(words))  # Output: apple (smallest alphabetically)
```

9. `max()`

- **Purpose:** Returns the largest item from an iterable or from two or more arguments.
- **Parameter(s):**
  - `arg1, arg2, ...` OR `iterable`: (Compulsory) Can be multiple arguments or a single iterable.
- **Description:** Finds the maximum value. Works with numbers, strings (lexicographic comparison), and other comparable data types.

```
# With iterable
numbers = [45, 12, 78, 23, 9, 56]
print(max(numbers))  # Output: 78

# With multiple arguments
print(max(100, 200, 50, 75, 150))  # Output: 200

# With strings (alphabetical order)
words = ["apple", "banana", "cherry", "date"]
print(max(words))  # Output: date (largest alphabetically)

# With mixed numeric types
mixed = [5, 10.5, 3, 8.2, 15]
print(max(mixed))  # Output: 15
```